📝 TASK 1: Introduction to Databases & SELECT Statement

1. What is a Database?

A **database** is an organized collection of data that allows information to be stored, managed, retrieved, and updated efficiently.

For example, a company's database can store:

- Employee information
- Customer information
- Product information
- Sales transactions
- Department information

SQL (Structured Query Language) is used to communicate with relational databases.

2. Create Departments Table

```
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);
```

Insert Data

```
INSERT INTO Departments VALUES
(1, 'Sales'),
(2, 'Marketing'),
(3, 'IT'),
(4, 'HR'),
(5, 'Finance');
```

Output

DepartmentID	DepartmentName
1	Sales
2	Marketing
3	IT
4	HR
5	Finance

---

3. Create Employees Table

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(100),
    DepartmentID INT,
    Salary DECIMAL(10,2),
    City VARCHAR(50),
    Experience INT
```

```
);
```

Insert Data

```
INSERT INTO Employees VALUES
(101, 'Rahul Sharma', 1, 55000, 'Delhi', 3),
(102, 'Priya Singh', 2, 62000, 'Mumbai', 4),
(103, 'Amit Kumar', 3, 75000, 'Bangalore', 6),
(104, 'Sneha Patel', 1, 58000, 'Ahmedabad', 3),
(105, 'Arjun Mehta', 4, 48000, 'Pune', 2),
(106, 'Neha Gupta', 5, 68000, 'Delhi', 5),
(107, 'Vikram Rao', 3, 82000, 'Hyderabad', 7),
(108, 'Anjali Verma', 2, 59000, 'Chennai', 3),
(109, 'Karan Singh', 1, 64000, 'Jaipur', 5),
(110, 'Pooja Shah', 5, 72000, 'Mumbai', 6);
```

---

4. Display All Records

```
SELECT *
FROM Employees;
```

Output

EmployeeID	EmployeeName	DepartmentID	Salary	City	Experience
101	Rahul Sharma	1	55000	Delhi	3
102	Priya Singh	2	62000	Mumbai	4
103	Amit Kumar	3	75000	Bangalore	6
104	Sneha Patel	1	58000	Ahmedabad	3
105	Arjun Mehta	4	48000	Pune	2
106	Neha Gupta	5	68000	Delhi	5
107	Vikram Rao	3	82000	Hyderabad	7
108	Anjali Verma	2	59000	Chennai	3
109	Karan Singh	1	64000	Jaipur	5
110	Pooja Shah	5	72000	Mumbai	6

Explanation:

The `SELECT *` statement displays **all columns and all records** from the Employees table.

---

5. Select Specific Columns

```
SELECT EmployeeID, EmployeeName, Salary
FROM Employees;
```

This displays only:

- Employee ID
- Employee Name
- Salary

EmployeeID	EmployeeName	Salary
101	Rahul Sharma	55000
102	Priya Singh	62000
103	Amit Kumar	75000
104	Sneha Patel	58000
105	Arjun Mehta	48000
106	Neha Gupta	68000
107	Vikram Rao	82000
108	Anjali Verma	59000
109	Karan Singh	64000
110	Pooja Shah	72000

---

6. Using Column Aliases

```
SELECT
    EmployeeName AS Name,
    Salary AS Monthly_Salary
FROM Employees;
```

Output

Name	Monthly_Salary
Rahul Sharma	55000
Priya Singh	62000
Amit Kumar	75000
Sneha Patel	58000
Arjun Mehta	48000
Neha Gupta	68000
Vikram Rao	82000
Anjali Verma	59000
Karan Singh	64000
Pooja Shah	72000

Explanation

`AS` gives a temporary alternative name to a column in the query output.

📝 TASK 2: WHERE, ORDER BY & Aggregate Functions

A. WHERE Clause

The `WHERE` clause is used to filter records according to a condition.

Find employees earning more than ₹60,000

```
SELECT *
FROM Employees
WHERE Salary > 60000;
```

EmployeeID	EmployeeName	DepartmentID	Salary	City	Experience
102	Priya Singh	2	62000	Mumbai	4
103	Amit Kumar	3	75000	Bangalore	6
106	Neha Gupta	5	68000	Delhi	5
107	Vikram Rao	3	82000	Hyderabad	7
109	Karan Singh	1	64000	Jaipur	5
110	Pooja Shah	5	72000	Mumbai	6

Find employees from Delhi

```
SELECT *
FROM Employees
WHERE City = 'Delhi';
```

EmployeeID	EmployeeName	DepartmentID	Salary	City	Experience
101	Rahul Sharma	1	55000	Delhi	3
106	Neha Gupta	5	68000	Delhi	5

Find employees with more than 3 years of experience

```
SELECT EmployeeName, Experience
FROM Employees
WHERE Experience > 3;
```

EmployeeName	Experience
Priya Singh	4
Amit Kumar	6
Neha Gupta	5
Vikram Rao	7
Karan Singh	5
Pooja Shah	6

B. Comparison Operators

Common comparison operators are:

Operator	Meaning
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal

Example:

```
SELECT *
FROM Employees
WHERE Salary >= 70000;
```

```
(107, 'Vikram Rao', 3, 82000, 'Hyderabad', 7),
```

Output

EmployeeID	EmployeeName	DepartmentID	Salary	City	Experience
103	Amit Kumar	3	75000	Bangalore	6
107	Vikram Rao	3	82000	Hyderabad	7
110	Pooja Shah	5	72000	Mumbai	6

C. ORDER BY

`ORDER BY` is used to sort records.

Sort Salary from Lowest to Highest

```
SELECT EmployeeName, Salary
FROM Employees
ORDER BY Salary ASC;
```

Output

EmployeeName	Salary
Arjun Mehta	48000
Rahul Sharma	55000
Sneha Patel	58000
Anjali Verma	59000
Priya Singh	62000
Karan Singh	64000
Neha Gupta	68000
Pooja Shah	72000
Amit Kumar	75000
Vikram Rao	82000

Sort Salary from Highest to Lowest

```
SELECT EmployeeName, Salary
FROM Employees
ORDER BY Salary DESC;
```

Output

EmployeeName	Salary
Vikram Rao	82000
Amit Kumar	75000
Pooja Shah	72000
Neha Gupta	68000
Karan Singh	64000
Priya Singh	62000
Anjali Verma	59000
Sneha Patel	58000
Rahul Sharma	55000
Arjun Mehta	48000

D. Aggregate Functions

SQL provides several aggregate functions.

COUNT()

Counts records.

```
SELECT COUNT(*) AS Total_Employees
FROM Employees;
```

Output

Total_Employees
10

SUM()

Calculates the total.

```
SELECT SUM(Salary) AS Total_Salary
FROM Employees;
```

Output

Total_Salary
643000

AVG()

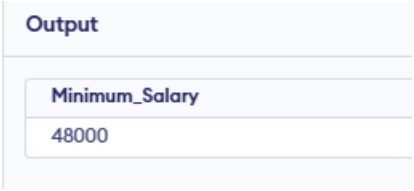
Calculates the average.

```
SELECT AVG(Salary) AS Average_Salary
FROM Employees;
```

MIN()

Finds the minimum value.

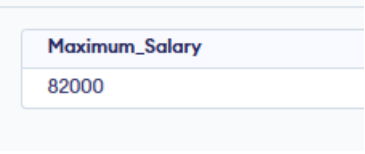
```
SELECT MIN(Salary) AS Minimum_Salary
FROM Employees;
```



MAX()

Finds the maximum value.

```
SELECT MAX(Salary) AS Maximum_Salary
FROM Employees;
```

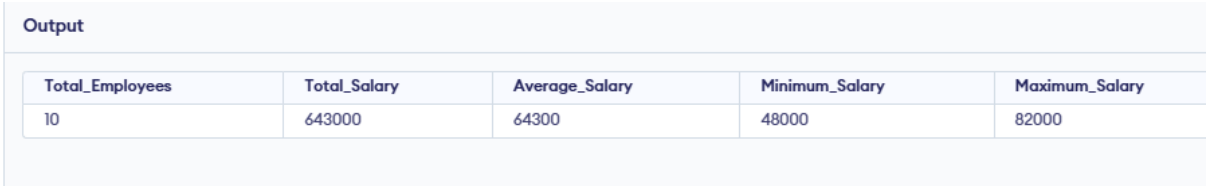


Combined Aggregate Query

```
SELECT
    COUNT(*) AS Total_Employees,
    SUM(Salary) AS Total_Salary,
    AVG(Salary) AS Average_Salary,
    MIN(Salary) AS Minimum_Salary,
    MAX(Salary) AS Maximum_Salary
FROM Employees;
```

Explanation

This query provides a summary of employee salaries using all five major aggregate functions.



Output

Total_Employees	Total_Salary	Average_Salary	Minimum_Salary	Maximum_Salary
10	643000	64300	48000	82000

📝 TASK 3: GROUP BY & HAVING

What is GROUP BY?

`GROUP BY` combines records having the same value and allows aggregate calculations for each group.

Average Salary by Department

```
SELECT
    DepartmentID,
    AVG(Salary) AS Average_Salary
FROM Employees
GROUP BY DepartmentID;
```

This calculates the average salary separately for each department.

Number of Employees in Each Department

```
SELECT
    DepartmentID,
    COUNT(*) AS Employee_Count
FROM Employees
GROUP BY DepartmentID;
```

Using HAVING

`HAVING` filters the results **after grouping**.

Departments having more than 1 employee

```
SELECT
    DepartmentID,
    COUNT(*) AS Employee_Count
FROM Employees
GROUP BY DepartmentID
HAVING COUNT(*) > 1;
```

Difference between WHERE and HAVING

WHERE filters individual rows before grouping.

HAVING filters groups after `GROUP BY`.

Example:

```
WHERE Salary > 60000
```

filters individual employees.

Whereas:

```
HAVING AVG(Salary) > 60000
```

filters departments based on their average salary.

📝 TASK 4: SQL JOINS

We have two related tables:

Employees

```
EmployeeID
EmployeeName
DepartmentID
Salary
City
Experience
```

Departments

```
DepartmentID
DepartmentName
```

The common column is:

DepartmentID

---

1. INNER JOIN

An `INNER JOIN` returns records that have matching values in both tables.

```
SELECT
    e.EmployeeID,
    e.EmployeeName,
    d.DepartmentName,
    e.Salary
FROM Employees e
INNER JOIN Departments d
ON e.DepartmentID = d.DepartmentID;
```

Purpose

It displays employees only when their department exists in the Departments table.

---

2. LEFT JOIN

A `LEFT JOIN` returns **all records from the left table** and matching records from the right table.

```
SELECT
    e.EmployeeID,
    e.EmployeeName,
    d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
ON e.DepartmentID = d.DepartmentID;
```

Purpose

All employees are displayed, even if their department does not have a matching record.

---

3. RIGHT JOIN

A `RIGHT JOIN` returns **all records from the right table** and matching records from the left table.

```
SELECT
    e.EmployeeID,
    e.EmployeeName,
    d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
```

```
ON e.DepartmentID = d.DepartmentID;
```

Purpose

All departments are displayed, even if no employee belongs to a particular department.

Note: Some database systems, such as certain SQLite setups, do not support `RIGHT JOIN`. If your SQL environment does not support it, use MySQL, PostgreSQL, SQL Server, or another DBMS that supports it.

📝 TASK 5: SQL SUBQUERIES

A **subquery** is a query inside another SQL query.

Problem 1: Employees Earning More Than Average Salary

```
SELECT EmployeeName, Salary
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees
);
```

Explanation

The inner query calculates the average salary.

The outer query finds employees whose salary is greater than that average.

Problem 2: Employee with Maximum Salary

```
SELECT EmployeeName, Salary
FROM Employees
WHERE Salary = (
    SELECT MAX(Salary)
    FROM Employees
);
```

Explanation

The inner query finds the maximum salary.

The outer query identifies the employee receiving that salary.

Problem 3: Employees in the Sales Department

```
SELECT EmployeeName
FROM Employees
WHERE DepartmentID = (
    SELECT DepartmentID
    FROM Departments
    WHERE DepartmentName = 'Sales'
);
```

This uses a subquery to find the department ID for Sales and then retrieves employees belonging to that department.

EXCEL TASKS

For Tasks 6–9, create an Excel workbook called:

Data_Analytics_Assignment.xlsx

Create these sheets:

```
1. Sales_Data
2. Products
3. Employees
4. Pivot_Table
5. Charts
```

TASK 6: Data Formatting, Sorting & Filtering

Use this sample Sales dataset:

Order ID	Date	Product	Category	Region	Salesperson	Quantity	Unit Price	Total Sales
O001	01-08-2026	Laptop	Electronics	North	Rahul	2	55000	110000
O002	02-08-2026	Mouse	Electronics	South	Priya	10	800	8000
O003	03-08-2026	Keyboard	Electronics	West	Amit	5	1500	7500
O004	04-08-2026	Monitor	Electronics	East	Sneha	3	12000	36000
O005	05-08-2026	Laptop	Electronics	South	Rahul	1	55000	55000
O006	06-08-2026	Chair	Furniture	North	Priya	4	7000	28000
O007	07-08-2026	Desk	Furniture	West	Amit	2	10000	20000
O008	08-08-2026	Headphones	Electronics	East	Sneha	6	2500	15000
O009	09-08-2026	Laptop	Electronics	North	Rahul	3	55000	165000
O010	10-08-2026	Chair	Furniture	South	Priya	5	7000	35000

Formatting

Apply:

- Bold headers
- Background color for headers
- Borders
- Currency format for Unit Price and Total Sales
- Date format for Date column
- Auto-fit columns
- Freeze the header row

Sorting

Sort Total Sales from:

Largest → Smallest

In Excel:

Data → Sort → Total Sales → Largest to Smallest

Filtering

Turn on filters:

Data → Filter

Example:

Filter:

```
Region = North
```

Or:

```
Total Sales > 50,000
```

Screenshot

Take screenshots showing:

1. Formatted dataset
2. Sorted dataset
3. Filtered dataset

📝 TASK 7: Conditional Formatting & Excel Functions — 15 Marks

A. Conditional Formatting

Select the **Total Sales** column.

Go to:

Home → Conditional Formatting → Highlight Cells Rules → Greater Than

Enter:

`50000`

This highlights sales greater than ₹50,000.

B. IF()

The `IF()` function checks a condition and returns one value if the condition is TRUE and another if it is FALSE.

Create a new column:

Performance

Formula:

```
=IF(I2>=50000,"High","Low")
```

Explanation

If Total Sales is greater than or equal to ₹50,000:

High

Otherwise:

Low

C. COUNTIF()

`COUNTIF()` counts cells that satisfy a condition.

Example:

```
=COUNTIF(E2:E11,"North")
```

This counts the number of sales transactions from the North region.

Another example:

```
=COUNTIF(D2:D11,"Electronics")
```

This counts Electronics transactions.

D. SUMIF()

`SUMIF()` adds values based on a condition.

Example:

```
=SUMIF(E2:E11,"North",I2:I11)
```

This calculates the total sales generated in the North region.

Summary

Function	Purpose
`IF()`	Checks a condition
`COUNTIF()`	Counts values matching a condition
`SUMIF()`	Adds values matching a condition

📝 TASK 8: VLOOKUP & XLOOKUP

Create a second table called **Products**.

Product	Category	Unit Price
Laptop	Electronics	55000
Mouse	Electronics	800
Keyboard	Electronics	1500
Monitor	Electronics	12000
Chair	Furniture	7000
Desk	Furniture	10000
Headphones	Electronics	2500

VLOOKUP()

Suppose cell `K2` contains:

```
Laptop
```

To find its price:

```
=VLOOKUP(K2,$A$2:$C$8,3,FALSE)
```

Explanation

`VLOOKUP()` searches for a value in the **first column** of a table and returns information from a specified column.

Here:

- `K2` = product to search
- `$A$2:$C$8` = lookup table
- `3` = return the third column
- `FALSE` = exact match

XLOOKUP()

The equivalent XLOOKUP formula is:

```
=XLOOKUP(K2,$A$2:$A$8,$C$2:$C$8)
```

Explanation

XLOOKUP searches one range and returns a corresponding value from another range.

VLOOKUP vs XLOOKUP

VLOOKUP	XLOOKUP
Older function	Newer function
Searches first column	Can search any lookup range
Uses column number	Uses return range
Less flexible	More flexible
Exact match requires `FALSE`	Exact match is the normal behavior

Example

If:

```
K2 = Laptop
```

Both formulas should return:

```
55000
```

📝 TASK 9: Pivot Table & Charts — 5 Marks

Create Pivot Table

Select the complete Sales dataset.

Go to:

Insert → PivotTable

Select:

New Worksheet

Recommended Pivot Table

Rows:
`Region`

Values:
`Total Sales`

This gives total sales by region.

Example structure:

Region	Sum of Total Sales
East	51,000
North	303,000
South	98,000
West	27,500

Your exact result should be generated from your workbook.

Another Useful Pivot Table

You can also use:

Rows:
`Category`

Values:
`Total Sales`

This tells you which category generates the highest sales.

Create a Chart

Select the Pivot Table.

Go to:

Insert → Column Chart

Choose:

Clustered Column Chart

Chart Title:

Total Sales by Region

Explanation

The chart visually compares sales performance across different regions. The region with the tallest column has the highest total sales, while the shortest column represents the lowest sales. This makes it easier for management to identify strong and weak-performing regions.

📄 FINAL REPORT STRUCTURE

Your PDF report can be organized like this:

```
DATA ANALYTICS USING SQL AND MICROSOFT EXCEL

1. Introduction
   - Objective
   - Tools Used

2. Task 1
   - Introduction to Databases
   - SELECT queries
   - Column aliases
   - Screenshots

3. Task 2
   - WHERE
   - ORDER BY
   - COUNT
   - SUM
   - AVG
   - MIN
   - MAX
   - Screenshots

4. Task 3
   - GROUP BY
   - HAVING
   - Screenshots
   - Explanation

5. Task 4
   - INNER JOIN
   - LEFT JOIN
   - RIGHT JOIN
```

   - Screenshots
   - Explanation

6. Task 5
   - Subqueries
   - Screenshots
   - Explanation

7. Task 6
   - Excel formatting
   - Sorting
   - Filtering
   - Screenshots

8. Task 7
   - Conditional Formatting
   - IF
   - COUNTIF
   - SUMIF
   - Screenshots

9. Task 8
   - VLOOKUP
   - XLOOKUP
   - Comparison
   - Screenshots

10. Task 9
    - Pivot Table
    - Chart
    - Explanation
    - Screenshot

11. Conclusion